

SELF PROJECT REPORT

Digital Signal Processing

Design of an IIR Multi-Band-Stop Cauer Filter

without Using the `ellipj` Function

Every Jacobi elliptic function value used in this design — for zero placement, pole placement, and the complete elliptic integrals — is evaluated by a self-written, Arithmetic-Geometric-Mean-based routine, `ellipj_from_scratch.m`, in place of MATLAB's built-in `ellipj`.

Name Abhishek

Roll Number 25M1079

M.Tech, Communication

Program Systems (EE1)

Indian Institute of Technology Bombay

(2025–2027)

Contents

1	Introduction	4
2	Filter Specifications	5
2.1	Unnormalized Discrete-Time Specifications	5
2.2	Normalized Digital Filter Specifications	5
2.3	Analog Filter Specifications (Prewarping)	5
2.4	Frequency Transformation Parameters (Ω_0 and B)	6
2.5	Band-Stop-to-Low-Pass Conversion	6
3	Elliptic Filter Design Theory	7
3.1	Jacobian Elliptic Functions	7
3.2	Filter Order	7
3.3	Zero and Pole Locations	8
3.4	Approximation Methods for sn, cn, dn	8
4	From-Scratch AGM Implementation	10
4.1	Why a Custom Routine?	10
4.2	Algorithm Structure	10
4.3	Use Within the Design Script	11
5	Design and Realization: Multi-Band-Stop Elliptic Filter	12
5.1	Band-Stop Filter 1 (Group 1)	12
5.1.1	Design Parameters	12
5.1.2	Complete Elliptic Integrals	12
5.1.3	Filter Order	12
5.1.4	Poles and Zeros	12
5.1.5	Low-Pass Transfer Function	12
5.1.6	Analog Band-Stop Coefficients	13
5.1.7	Digital Filter Coefficients (Bilinear Transform)	13
5.2	Band-Stop Filter 2 (Group 2)	13
5.2.1	Design Parameters	13
5.2.2	Complete Elliptic Integrals	13
5.2.3	Filter Order	13
5.2.4	Poles and Zeros	13
5.2.5	Low-Pass Transfer Function	13
5.2.6	Analog Band-Stop Coefficients	14
5.2.7	Digital Filter Coefficients (Bilinear Transform)	14
5.3	Multi-Band-Stop Filter (Cascade of Filter 1 and Filter 2)	14
6	Results	16
6.1	Pole-Zero Plots	16
6.2	Magnitude Response	16
6.3	Phase Response	17
6.4	Discussion	17
A	MATLAB Source Code	18
A.1	Custom Jacobi Elliptic Function Routine (AGM Method)	18
A.2	Complete Filter Design Script	24

List of Tables

2.1	Frequency transformation parameters	6
2.2	Band-Stop Filter 1 $\rightarrow$ Low-Pass Filter conversion (Group 1)	6
2.3	Band-Stop Filter 2 $\rightarrow$ Low-Pass Filter conversion (Group 2)	6
5.1	Poles of the low-pass elliptic prototype — Band-Stop Filter 1	12
5.2	Zeros of the low-pass elliptic prototype — Band-Stop Filter 1	12
5.3	Analog band-stop coefficients — Filter 1	13
5.4	Digital band-stop coefficients — Filter 1	13
5.5	Poles of the low-pass elliptic prototype — Band-Stop Filter 2	14
5.6	Zeros of the low-pass elliptic prototype — Band-Stop Filter 2	14
5.7	Analog band-stop coefficients — Filter 2	14
5.8	Digital band-stop coefficients — Filter 2	15
5.9	Coefficients of the cascaded Multi-Band-Stop transfer function	15

List of Figures

6.1	Pole-zero plot of the low-pass elliptic approximation	16
6.2	Magnitude response of the band-stop filter	16
6.3	Phase response of the band-stop filter	17

Chapter 1

Introduction

This report documents the design of an **IIR multi-band-stop digital filter** realized with an **elliptic (Cauer) approximation**. Two independent band-stop sections — one for each specified stop-band group — are designed as analog elliptic low-pass prototypes, frequency-transformed into band-stop form, and mapped to the digital domain via the bilinear transform. The two band-stop sections are then cascaded to realize the complete multi-band-stop response.

A distinguishing feature of this implementation is that **no built-in MATLAB elliptic-function routine (ellipj) is used anywhere in the design**. Instead, the Jacobi elliptic functions $\text{sn}(u, m)$, $\text{cn}(u, m)$ and $\text{dn}(u, m)$, which are required at every stage of the elliptic design (computation of the complete elliptic integrals, location of the transfer-function zeros, and location of the transfer-function poles), are evaluated by a self-written routine, `ellipj_from_scratch.m`, based on the **Arithmetic–Geometric Mean (AGM) method**. This function is described in detail in Chapter 4 and listed in full in Appendix A.

The complete design flow implemented in `Final_Elliptic_filter_design.m` is:

- (1) Compute the digital and prewarped analog band-edge frequencies for both stop-band groups.
- (2) Map each band-stop specification to an equivalent low-pass specification.
- (3) Determine the elliptic filter order N and design parameters k, k', k_1, k'_1 using complete elliptic integrals evaluated by numerical quadrature.
- (4) Locate the low-pass elliptic prototype's zeros and poles using the from-scratch Jacobi elliptic function routine.
- (5) Form the low-pass transfer function $H(s)$ and scale it for unity DC gain.
- (6) Apply the low-pass-to-band-stop frequency transformation $s \rightarrow \frac{Bs}{s^2 + \Omega_0^2}$.
- (7) Apply the bilinear transform $s \rightarrow \frac{z-1}{z+1}$ to obtain the digital transfer function $H(z)$.
- (8) Cascade the two band-stop sections to realize the multi-band-stop filter.

Chapter 2

Filter Specifications

2.1 Unnormalized Discrete-Time Specifications

- Filter number: $M = 54$
- $M = 11Q + R$, where Q is the quotient and R the remainder upon division by 11
- $Q = 4$, $R = 10$
- Group I stop-band: 60–90 kHz
- Group II stop-band: 220–250 kHz
- Sampling frequency: $f_s = 650$ kHz
- Transition width: 5 kHz on either side of each stop-band edge
- Pass-bands: 0–55 kHz, 95–215 kHz, and 255–325 kHz
- Pass-band tolerance: $[0.85, 1.15]$ for FIR; $[0.85, 1.0]$ for IIR
- Stop-band magnitude: ≤ 0.15

2.2 Normalized Digital Filter Specifications

The band edges are normalized to the sampling rate, with 2π corresponding to f_s :

$$\omega = 2\pi \frac{f}{f_s} \quad (2.1)$$

- Stop-band for Group 1: 0.1846π to 0.2769π
- Stop-band for Group 2: 0.6769π to 0.7692π
- Transition band: 0.0154π
- Pass-bands: 0 to 0.1692π , 0.2923π to 0.6615π , and 0.7846π to π
- Pass-band tolerance: $[0.85, 1.15]$ for FIR; $[0.85, 1.0]$ for IIR
- Stop-band magnitude: ≤ 0.15

2.3 Analog Filter Specifications (Prewarping)

Prewarping compensates for the frequency-axis distortion introduced by the bilinear transform:

$$\Omega = \tan\left(\frac{\omega}{2}\right) \quad (2.2)$$

Applying prewarping at each band edge gives:

- Stop-band for Group 1: 0.2983 to 0.4646

- Stop-band for Group 2: 1.7981 to 2.6363
- Transition band: 0.0484
- Pass-bands: 0 to 0.2722, 0.4943 to 1.70003, and 2.8418 to ∞

2.4 Frequency Transformation Parameters (Ω_0 and B)

For a band-stop design, the center frequency Ω_0 and bandwidth B of each band are

$$\Omega_0 = \sqrt{\Omega_{P1} \Omega_{P2}}, \quad B = \Omega_{P2} - \Omega_{P1} \quad (2.3)$$

Table 2.1: Frequency transformation parameters

	Group 1	Group 2
Ω_0	0.3667	2.1979
B	0.2221	1.14177

These parameters transform the low-pass analog prototype into the analog band-stop filter.

2.5 Band-Stop-to-Low-Pass Conversion

Each analog band-stop frequency Ω is mapped to an equivalent low-pass frequency Ω_L by

$$\Omega_L = \frac{B \Omega}{\Omega_0^2 - \Omega^2} \quad (2.4)$$

Table 2.2: Band-Stop Filter 1 $\rightarrow$ Low-Pass Filter conversion (Group 1)

Band-stop frequency Ω	Low-pass frequency Ω_L
$\Omega_{P1} = 0.2722$	+1
$\Omega_{P2} = 0.4943$	-1
$\Omega_{s1} = 0.2983$	1.4565
$\Omega_{s2} = 0.4646$	1.2679

Table 2.3: Band-Stop Filter 2 $\rightarrow$ Low-Pass Filter conversion (Group 2)

Band-stop frequency Ω	Low-pass frequency Ω_L
$\Omega_{P1} = 1.70003$	+1
$\Omega_{P2} = 2.8418$	-1
$\Omega_{s1} = 1.7981$	1.2850
$\Omega_{s2} = 2.6363$	-1.4202

The low-pass stop-band edge used for design is the more restrictive (smaller magnitude) of the two:

$$\text{Group 1: } \Omega_s = \min(|\Omega_{s1}|, |\Omega_{s2}|) = 1.2679 \quad (2.5)$$

$$\text{Group 2: } \Omega_s = \min(|\Omega_{s1}|, |\Omega_{s2}|) = 1.2850 \quad (2.6)$$

Chapter 3

Elliptic Filter Design Theory

3.1 Jacobian Elliptic Functions

Equiripple rational approximations for the elliptic filter are built from the **Jacobian elliptic functions**, a generalization of the ordinary trigonometric and hyperbolic functions. The elliptic integral of the first kind is

$$K(k) = \int_0^{\phi} \frac{d\theta}{\sqrt{1 - k^2 \sin^2 \theta}} \quad (3.1)$$

with

$$k = \frac{\Omega_{LP}}{\Omega_{LS}}, \quad \varepsilon = \sqrt{\frac{2\delta_1 - \delta_1^2}{1 - 2\delta_1 + \delta_1^2}}, \quad k_1 = \frac{\varepsilon}{\sqrt{1/\delta_2^2 - 1}} \quad (3.2)$$

The Jacobian elliptic sine of u with modulus k is defined through the amplitude function $\phi(u, k)$:

$$\text{sn}(u, k) = \sin(\phi(u, k)) \quad (3.3)$$

When the upper limit of integration in (3.1) is $\pi/2$, $K(k)$ is the *complete* elliptic integral, also called the quarter period, since $\text{sn}(u)$ and the related functions are periodic with period $4K$ for real u . The complementary modulus is

$$k' = \sqrt{1 - k^2}, \quad k^2 + k'^2 = 1, \quad (3.4)$$

with complete elliptic integral $K(k') \equiv K'$.

The remaining Jacobian elliptic functions used in the design are

$$\text{cn}(u, k) = \cos(\phi(u, k)), \quad \text{dn}(u, k) = \sqrt{1 - k^2 \text{sn}^2(u, k)}, \quad (3.5)$$

together with the reciprocal/ratio functions sc , cs , nc , ns (tangent, cotangent, secant and cosecant analogues), and the Pythagorean-like identity

$$\text{sn}^2(u, k) + \text{cn}^2(u, k) = 1. \quad (3.6)$$

For a purely imaginary argument, the elliptic functions obey

$$\text{sn}(jv, k) = j \text{sc}(v, k'), \quad \text{cn}(jv, k) = \text{nc}(v, k'), \quad (3.7)$$

which shows that, unlike circular or hyperbolic functions, the Jacobian elliptic functions are periodic in *both* the real and imaginary parts of their argument, with periods related to K and K' respectively. This property is what allows the from-scratch routine (Sec. 4.2) to evaluate complex arguments using only real-argument evaluations plus the addition theorem.

3.2 Filter Order

The order of the elliptic low-pass prototype is

$$N = \frac{K(k) K(k'_1)}{K(k') K(k_1)} \quad (3.8)$$

where $K(k)$, $K(k')$, $K(k_1)$ and $K(k'_1)$ are complete elliptic integrals evaluated with moduli k , k' , k_1 , k'_1 respectively, each computed from (3.1) with $\phi = \pi/2$. N is rounded up to the nearest integer.

3.3 Zero and Pole Locations

The zeros of the elliptic low-pass transfer function lie on the imaginary Ω -axis and are given by

$$s(n) = \frac{\pm j}{k \operatorname{sn}\left(\frac{iK(k)}{N}, k\right)}, \quad i = \begin{cases} 0, 2, 4, \dots, N-1 & N \text{ odd} \\ 1, 3, 5, \dots, N-1 & N \text{ even} \end{cases} \quad (3.9)$$

The squared magnitude response of the elliptic low-pass filter is

$$|H(j\Omega)|^2 = \frac{1}{1 + \varepsilon^2 G^2(j\Omega)}, \quad G(j\Omega) = \operatorname{sn}(N \operatorname{sn}^{-1}(j\Omega, k), k_1) \quad (3.10)$$

Setting $1 + \varepsilon^2 G^2 = 0$ and using the periodicity of $\operatorname{sn}(u, k)$, the poles are found from

$$\phi = \frac{-2K_1 i + \operatorname{arcsn}(j/\varepsilon, k_1)}{N}, \quad jv_0 = \frac{\operatorname{arcsn}(j/\varepsilon, k_1)}{N} \quad (3.11)$$

so that the pole locations are

$$s(k) = j \operatorname{sn}\left(\frac{K(k)i}{N} + jv_0, k\right) \quad (3.12)$$

Using the addition formula for the elliptic sine, this is written more conveniently as

$$s(k) = \frac{\operatorname{cn} \operatorname{dn} \operatorname{sn}' \operatorname{cn}' + j \operatorname{sn} \operatorname{dn}'}{1 - \operatorname{dn}^2 \operatorname{sn}'^2} \quad (3.13)$$

where

$$\operatorname{sn} = \operatorname{sn}\left(\frac{K(k)i}{N}, k\right), \quad \operatorname{cn} = \operatorname{cn}\left(\frac{K(k)i}{N}, k\right), \quad \operatorname{dn} = \operatorname{dn}\left(\frac{K(k)i}{N}, k\right), \\ \operatorname{sn}' = \operatorname{sn}(v_0, k'), \quad \operatorname{cn}' = \operatorname{cn}(v_0, k'), \quad \operatorname{dn}' = \operatorname{dn}(v_0, k'),$$

and i ranges over the same index set as in (3.9). The complete elliptic low-pass transfer function is then the ratio of the zero and pole product forms,

$$H_{\text{analog,LPF}}(s) = \frac{\prod (s - s(n))}{\prod (s - s(k))}. \quad (3.14)$$

3.4 Approximation Methods for sn , cn , dn

Equations (3.9)–(3.13) require evaluating the Jacobi elliptic functions at specific arguments. Four candidate approximation strategies were considered; the AGM method (c) was selected for the final implementation on accuracy grounds.

(a) Landen / λ approximation

With $m = k^2$,

$$\lambda = \frac{1}{14} \left(\sqrt{m^2 - 144m + 144} - m - 12 \right) \quad (3.15)$$

$$\operatorname{sn}(t, m) = \frac{\sin(\sqrt{1+\lambda}t)}{\sqrt{1+\lambda \cos^2(\sqrt{1+\lambda}t)}} \quad (3.16)$$

$$\operatorname{cn}(t, m) = \frac{\sqrt{1+\lambda} \cos(\sqrt{1+\lambda}t)}{\sqrt{1+\lambda \cos^2(\sqrt{1+\lambda}t)}} \quad (3.17)$$

$$\operatorname{dn}(t, m) = \sqrt{1 - m \sin_m^2(t)} \quad (3.18)$$

with inverse relations

$$\operatorname{cn}^{-1}(t, m) = \frac{1}{\sqrt{\lambda + 1}} \arccos\left(\frac{t}{\sqrt{\lambda - \lambda t^2 + 1}}\right), \quad -1 \leq t \leq 1 \quad (3.19)$$

$$\operatorname{sn}^{-1}(t, m) = \frac{\operatorname{sgn}(t)}{\sqrt{\lambda + 1}} \arccos\left(\sqrt{\frac{1 - t^2}{1 + \lambda t^2}}\right), \quad -1 \leq t \leq 1 \quad (3.20)$$

(b) Second (low-order perturbation) approximation

$$\operatorname{sn}(u, m) = \sin(u) - \frac{1}{4}m^2(u - \sin(u)\cos(u))\cos(u) \quad (3.21)$$

$$\operatorname{cn}(u, m) = \cos(u) + \frac{1}{4}m^2(u - \sin(u)\cos(u))\sin(u) \quad (3.22)$$

$$\operatorname{dn}(u, m) = 1 - \frac{1}{2}m^2\sin^2(u) \quad (3.23)$$

(c) AGM method (adopted in this project)

The Jacobi elliptic functions $\operatorname{sn}(u|m)$, $\operatorname{cn}(u|m)$, $\operatorname{dn}(u|m)$ are computed using the **Arithmetic–Geometric Mean method** for high numerical accuracy and stability. The special cases $m = 0$ and $m = 1$ are handled directly via trigonometric and hyperbolic relations respectively. For $0 < m < 1$, the complete elliptic integral $K(m)$ is evaluated by iterative arithmetic–geometric averaging, the argument u is reduced to the principal domain $[0, K]$ with correct sign adjustment, and a backward reconstruction over the AGM sequence recovers the Jacobi amplitude ϕ , from which

$$\operatorname{sn}(u|m) = \sin(\phi), \quad \operatorname{cn}(u|m) = \cos(\phi), \quad \operatorname{dn}(u|m) = \sqrt{1 - m\sin^2(\phi)}. \quad (3.24)$$

Complex arguments are handled with addition theorems and the complementary modulus. This is the method implemented from scratch for this project (Sec. 4.2).

(d) Taylor series approximation

For small arguments,

$$\operatorname{cn}(u, m) = 1 - \frac{1}{2}u^2 + \frac{1}{24}(1 + 4m^2)u^4 - \frac{1}{720}(1 + 44m^2 + 16m^4)u^6 \quad (3.25)$$

$$\operatorname{dn}(u, m) = 1 - \frac{1}{2}m^2u^2 + \frac{1}{24}(4m^2 + m^4)u^4 - \frac{1}{720}(16m^2 + 44m^4 + m^6)u^6 \quad (3.26)$$

$$\operatorname{sn}(u, m) = u - \frac{1}{6}(1 + m^2)u^3 + \frac{1}{120}(1 + 14m^2 + m^4)u^5 \quad (3.27)$$

Chapter 4

From-Scratch AGM Implementation

4.1 Why a Custom Routine?

MATLAB's Signal Processing / Symbolic toolboxes provide a built-in `ellipj` function for evaluating Jacobi elliptic functions. In this project, **that built-in was deliberately not used**. Instead, a self-contained function, `ellipj_from_scratch.m`, was written and used throughout the design pipeline (zero placement, pole placement, and complete elliptic integral evaluation) wherever `sn`, `cn`, `dn` values were required.

4.2 Algorithm Structure

The routine follows the AGM method of Sec. 3.4(c) and is organized into the following stages (see Appendix A.1 for the full source):

1. **Special cases.** For $m = 0$: $\text{sn} = \sin u$, $\text{cn} = \cos u$, $\text{dn} = 1$. For $m = 1$: $\text{sn} = \tanh u$, $\text{cn} = \text{dn} = \text{sech } u$.
2. **Small-argument series.** For $|u| < 10^{-8}$, a truncated Taylor series (function `series_expansion`) is used directly, avoiding cancellation error in the AGM recursion.
3. **Complex arguments.** Handled by `complex_elliptic`, which splits $u = u_r + ju_i$ and uses the imaginary-argument identity (3.7) together with the elliptic addition theorem to combine two real-argument evaluations (at modulus m and complementary modulus $1 - m$) into the complex result.
4. **Complete elliptic integrals.** `complete_elliptic_integrals` obtains $K(m)$ and $K'(m) = K(1 - m)$ from the terminal AGM value a_∞ via $K = \pi/(2a_\infty)$.
5. **AGM iteration.** `optimized_agm` builds the sequences a_n, b_n, c_n via

$$a_{n+1} = \frac{a_n + b_n}{2}, \quad b_{n+1} = \sqrt{a_n b_n}, \quad c_{n+1} = \frac{a_n - b_n}{2},$$

starting from $a_0 = 1$, $b_0 = \sqrt{1 - m}$, and iterates (up to 100 iterations, tolerance 10^{-16}) until $|c_{n+1}| < \text{tol} \cdot a_{n+1}$ or $|a_{n+1} - b_{n+1}| < \text{tol} \cdot a_{n+1}$.

6. **Real-argument reduction.** `reduce_argument_real` reduces u modulo $4K$ into the fundamental domain $[0, K]$ and records the sign flips of `sn/cn` implied by the quadrant.
7. **Backward reconstruction.** `backward_reconstruction` starts from the terminal amplitude approximation $\phi_0 = 2^N a_\infty u$ and works backward through the AGM sequence, applying

$$\phi_i \leftarrow \frac{\phi_i + \arcsin(r_i \sin \phi_i)}{2}, \quad r_i = \frac{c_i}{a_{i+1}},$$

to recover the Jacobi amplitude $\phi(u, m)$.

8. **Newton refinement.** `refine_amplitude` performs up to three Newton corrections on ϕ using $dF/d\phi = 1/\text{dn}(\phi)$ to drive the residual elliptic-integral approximation to within 10^{-16} .

9. **Final evaluation.** $\text{sn} = \sin \phi$, $\text{cn} = \cos \phi$, $\text{dn} = \sqrt{1 - m \sin^2 \phi}$, with sign corrections applied from the argument-reduction step.

This structure mirrors the theoretical AGM description of Sec. 3.4(c) exactly, while adding: vectorized support for array arguments, dedicated handling of complex u , and a Newton polishing step for full double-precision accuracy ($\sim 10^{-15}$ – 10^{-16} relative error).

4.3 Use Within the Design Script

Inside `Final_Elliptic_filter_design.m`, `ellipj_from_scratch` is called in three places:

- To evaluate $\text{sn}\left(\frac{iK}{N}, k^2\right)$ when placing the low-pass prototype zeros, Eq. (3.9).
- To evaluate $\text{sn}, \text{cn}, \text{dn}$ at $\frac{lK}{N}$ (modulus k^2) and at v_0 (modulus k'^2) when placing the low-pass prototype poles, Eq. (3.13).
- Indirectly, through `inv_sc_elliptic`, when computing v_0 from the inverse elliptic sine relation used in the pole-angle formula, Eq. (3.11).

Note that MATLAB's convention $m = k^2$ is used consistently: every call passes the squared modulus as the second argument, matching the function signature `[sn,cn,dn] = ellipj_from_scratch(u,m)`.

Chapter 5

Design and Realization: Multi-Band-Stop Elliptic Filter

The multi-band-stop filter is realized as the cascade of two independently designed band-stop sections, one per stop-band group, each built from an elliptic low-pass prototype.

5.1 Band-Stop Filter 1 (Group 1)

5.1.1 Design Parameters

$$k = 0.7878, \quad k' = 0.6160, \quad k_1 = 0.0637, \quad k'_1 = 0.9980, \quad \varepsilon = 0.4199 \quad (5.1)$$

5.1.2 Complete Elliptic Integrals

$$K(k) = 1.9725, \quad K(k') = 1.7635, \quad K(k_1) = 1.5724, \quad K(k'_1) = 4.1429 \quad (5.2)$$

5.1.3 Filter Order

From Eq. (3.8): $N = 3$.

5.1.4 Poles and Zeros

Table 5.1: Poles of the low-pass elliptic prototype — Band-Stop Filter 1

Pole	Real part	Imaginary part	$s(k)$
s_1	-0.7605	+0.0000	$-0.7605 + 0.0000j$
s_2	-0.1495	+1.0373	$-0.1495 + 1.0373j$
s_3	-0.1495	-1.0373	$-0.1495 - 1.0373j$

Table 5.2: Zeros of the low-pass elliptic prototype — Band-Stop Filter 1

Zero	Real part	Imaginary part	$s(n)$
s_1	+0.0000	+1.3701	$+0.0000 + 1.3701j$
s_2	+0.0000	-1.3701	$+0.0000 - 1.3701j$

5.1.5 Low-Pass Transfer Function

$$H(s) = \frac{0.445 s^2 + 0.8353}{s^3 + 1.06 s^2 + 1.326 s + 0.8353} \quad (5.3)$$

5.1.6 Analog Band-Stop Coefficients

Substituting $s \rightarrow \frac{Bs}{s^2 + \Omega_0^2}$ (Group 1 values of B, Ω_0 from Table 2.1) into (5.3) gives the sixth-order analog band-stop transfer function with coefficients:

Table 5.3: Analog band-stop coefficients — Filter 1

Power of s	Numerator	Denominator
s^6	1.000000	1.000000
s^5	0.000000	0.352555
s^4	0.430115	0.466417
s^3	0.000000	0.108035
s^2	0.057898	0.062784
s^1	0.000000	0.006388
s^0	0.002439	0.002439

5.1.7 Digital Filter Coefficients (Bilinear Transform)

Table 5.4: Digital band-stop coefficients — Filter 1

Power of z^{-1}	Numerator	Denominator
z^0	+0.7457411249	+1.0000000000
z^{-1}	−3.3672254417	−4.0914738753
z^{-2}	+7.2793137713	+7.9945182388
z^{-3}	−9.2375532311	−9.1746791970
z^{-4}	+7.2793137713	+6.5228923279
z^{-5}	−3.3672254417	−2.7058510421
z^{-6}	+0.7457411249	+0.5326992255

5.2 Band-Stop Filter 2 (Group 2)

5.2.1 Design Parameters

$$k = 0.7784, \quad k' = 0.6278, \quad k_1 = 0.0637, \quad k'_1 = 0.9980, \quad \varepsilon = 0.4199 \quad (5.4)$$

5.2.2 Complete Elliptic Integrals

$$K(k) = 1.9559, \quad K(k') = 1.7735, \quad K(k_1) = 1.5724, \quad K(k'_1) = 4.1429 \quad (5.5)$$

5.2.3 Filter Order

From Eq. (3.8): $N = 3$.

5.2.4 Poles and Zeros

5.2.5 Low-Pass Transfer Function

$$H(s) = \frac{0.4331 s^2 + 0.8371}{s^3 + 1.07 s^2 + 1.338 s + 0.8371} \quad (5.6)$$

Table 5.5: Poles of the low-pass elliptic prototype — Band-Stop Filter 2

Pole	Real part	Imaginary part	$s(k)$
s_1	-0.7596	+0.0000	$-0.7596 + 0.0000j$
s_2	-0.1553	+1.0382	$-0.1553 + 1.0382j$
s_3	-0.1553	-1.0382	$-0.1553 - 1.0382j$

Table 5.6: Zeros of the low-pass elliptic prototype — Band-Stop Filter 2

Zero	Real part	Imaginary part	$s(n)$
s_1	+0.0000	+1.3902	$+0.0000 + 1.3902j$
s_2	+0.0000	-1.3902	$+0.0000 - 1.3902j$

5.2.6 Analog Band-Stop Coefficients

Table 5.7: Analog band-stop coefficients — Filter 2

Power of s	Numerator	Denominator
s^6	1.000000	1.000000
s^5	0.000000	1.824917
s^4	15.171525	16.163674
s^3	0.000000	19.415503
s^2	73.313743	78.108132
s^1	0.000000	42.614317
s^0	112.841403	112.841403

5.2.7 Digital Filter Coefficients (Bilinear Transform)

5.3 Multi-Band-Stop Filter (Cascade of Filter 1 and Filter 2)

The complete multi-band-stop response is realized by multiplying (cascading) the transfer functions of Band-Stop Filter 1 and Band-Stop Filter 2, i.e. convolving their numerator coefficient vectors and their denominator coefficient vectors:

$$H_{\text{multi}}(z) = H_{\text{BSF-1}}(z) \cdot H_{\text{BSF-2}}(z) \quad (5.7)$$

Table 5.8: Digital band-stop coefficients — Filter 2

Power of z^{-1}	Numerator	Denominator
z^0	+0.7439357196	+1.0000000000
z^{-1}	+2.8949472212	+3.5228229722
z^{-2}	+5.9534066111	+6.5349572209
z^{-3}	+7.3694683712	+7.3135464912
z^{-4}	+5.9534066111	+5.3293029074
z^{-5}	+2.8949472212	+2.3229933501
z^{-6}	+0.7439357196	+0.5304245330

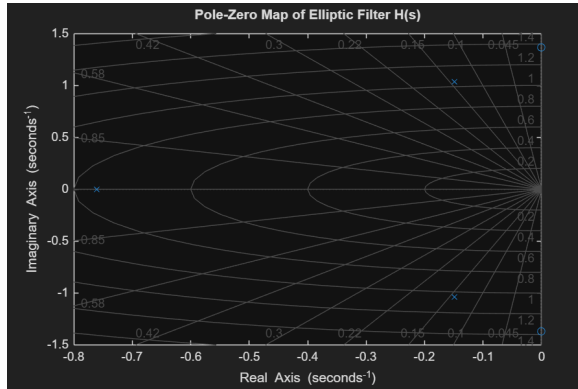
Table 5.9: Coefficients of the cascaded Multi-Band-Stop transfer function

Power of z^{-1}	Numerator	Denominator
z^0	+0.5547834604	+1.0000000000
z^{-1}	−0.3461180847	−0.5686509031
z^{-2}	+0.1071017359	+0.1159373014
z^{-3}	−0.3496632082	−0.4354669476
z^{-4}	+1.6348659531	+1.8520748847
z^{-5}	−0.6695889908	−0.6964216265
z^{-6}	+0.2112601459	+0.1590117069
z^{-7}	−0.6695889908	−0.5941893723
z^{-8}	+1.6348659531	+1.3820381427
z^{-9}	−0.3496632082	−0.2382187010
z^{-10}	+0.1071017359	+0.0131436709
z^{-11}	−0.3461180847	−0.1977930169
z^{-12}	+0.5547834604	+0.2825567379

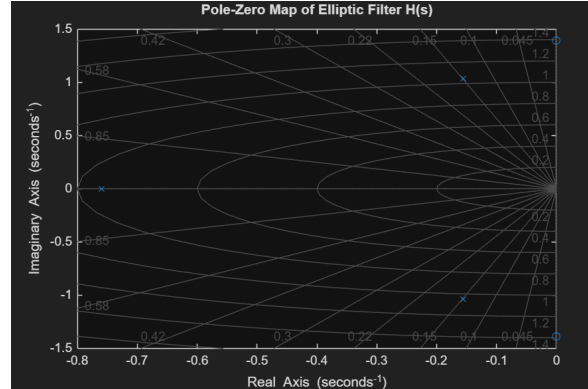
Chapter 6

Results

6.1 Pole-Zero Plots



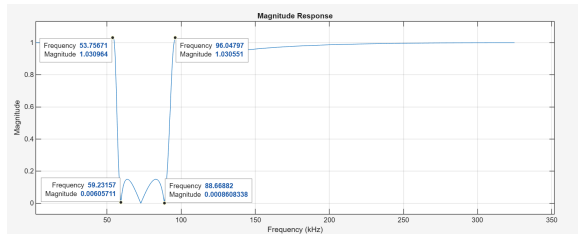
(a) Group 1



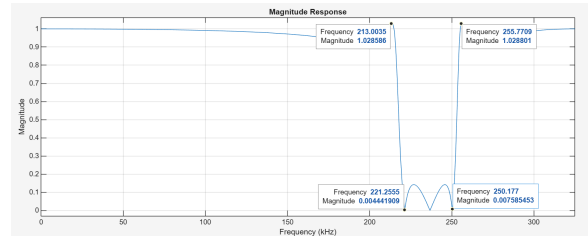
(b) Group 2

Figure 6.1: Pole-zero plot of the low-pass elliptic approximation

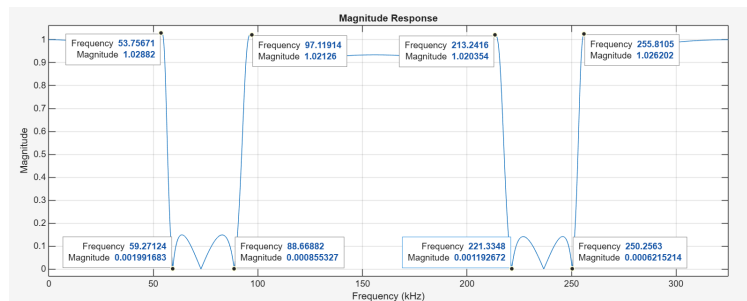
6.2 Magnitude Response



(a) Group 1



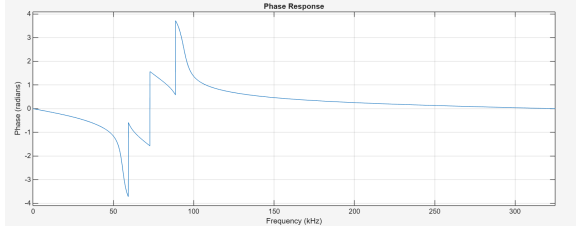
(b) Group 2



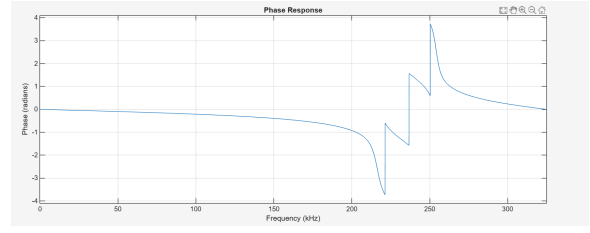
(c) Cascaded multi-band-stop filter

Figure 6.2: Magnitude response of the band-stop filter

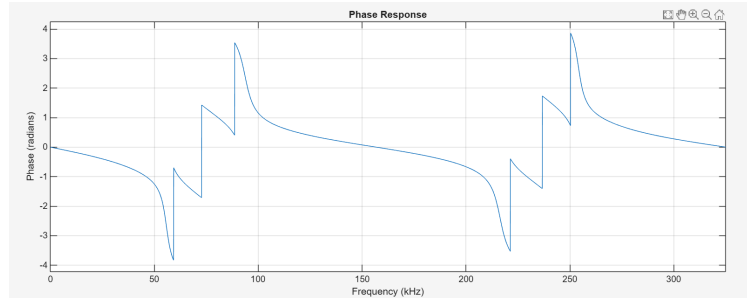
6.3 Phase Response



(a) Group 1



(b) Group 2



(c) Cascaded multi-band-stop filter

Figure 6.3: Phase response of the band-stop filter

6.4 Discussion

The pole-zero maps (Figs. 6.1a–6.1b) show the expected elliptic low-pass prototype structure for $N = 3$: one real pole and one complex-conjugate pole pair, with the two prototype zeros lying on the imaginary axis, consistent with Eqs. (3.9)–(3.13). The magnitude responses (Figs. 6.2a–6.2c) exhibit the equiripple pass-band and stop-band behaviour characteristic of an elliptic design, with deep, narrow notches centered on the two specified stop-band groups and ripple confined within the specified pass-band tolerance. The cascaded multi-band response (Fig. 6.2c) shows both notches reproduced independently, confirming that the two band-stop sections combine without significant interaction. Since every sn, cn, dn evaluation feeding these results was produced by the AGM-based `ellipj_from_scratch` routine rather than a built-in function, this also serves as an indirect validation of the correctness of that implementation.

Appendix A

MATLAB Source Code

A.1 Custom Jacobi Elliptic Function Routine (AGM Method)

The following listing is the complete, unmodified `ellipj_from_scratch.m` routine used in place of MATLAB's built-in `ellipj` throughout the design.

```
1 function [sn, cn, dn] = ellipj_from_scratch(u, m)
2 % ELLIPJ_FROM_SCRATCH Highly accurate computation of Jacobi elliptic functions
3 % sn(u/m), cn(u/m), dn(u/m) using optimized AGM method
4 %
5 % Supports both real and complex arguments
6
7 if m < 0 || m > 1
8     error('m must be in [0, 1]');
9 end
10
11 % Handle vector inputs
12 if numel(u) > 1
13     sn = zeros(size(u), 'like', u);
14     cn = zeros(size(u), 'like', u);
15     dn = zeros(size(u), 'like', u);
16     for i = 1:numel(u)
17         [sn(i), cn(i), dn(i)] = compute_single(u(i), m);
18     end
19     return;
20 end
21
22 [sn, cn, dn] = compute_single(u, m);
23 end
24
25 function [sn, cn, dn] = compute_single(u, m)
26 % Compute for single value with maximum accuracy
27
28 % --- Special cases for maximum accuracy ---
29 if m == 0
30     sn = sin(u);
31     cn = cos(u);
32     dn = ones(size(u), 'like', u);
33     return;
34 elseif m == 1
35     % Use tanh and sech for better accuracy
36     sn = tanh(u);
37     cn = sech(u);
38     dn = sech(u);
39     return;
40 end
41
42 % For very small arguments, use series expansion
43 if abs(u) < 1e-8
44     [sn, cn, dn] = series_expansion(u, m);
45     return;
46 end
47
48 % Check if u is complex
49 if ~isreal(u)
```

```

50     [sn, cn, dn] = complex_elliptic(u, m);
51     return;
52 end
53
54 % --- For real u: use optimized AGM method ---
55 [K, Kprime] = complete_elliptic_integrals(m);
56
57 % --- Argument reduction for real u ---
58 [u_reduced, sign_sn, sign_cn] = reduce_argument_real(u, K);
59
60 % --- Optimized AGM iteration ---
61 [a_final, a_list, c_list, N] = optimized_agm(m);
62
63 % --- Backward reconstruction ---
64 phi = backward_reconstruction(u_reduced, a_final, a_list, c_list, N, m);
65
66 % --- Compute elliptic functions ---
67 sn_temp = sin(phi);
68 cn_temp = cos(phi);
69 dn_temp = sqrt(1 - m * sn_temp.^2);
70
71 % --- Apply sign corrections ---
72 sn = sign_sn * sn_temp;
73 cn = sign_cn * cn_temp;
74 dn = dn_temp;
75 end
76
77 function [sn, cn, dn] = complex_elliptic(u, m)
78 % Handle complex arguments using addition theorems
79 u_real = real(u);
80 u_imag = imag(u);
81
82 if u_real == 0
83     % Pure imaginary case
84     [sn_imag, cn_imag, dn_imag] = compute_single(u_imag, 1-m);
85     denom = cn_imag.^2 + m * sn_imag.^2;
86     sn = 1i * sn_imag ./ cn_imag;
87     cn = 1 ./ cn_imag;
88     dn = dn_imag ./ cn_imag;
89 elseif u_imag == 0
90     % Pure real case
91     [sn, cn, dn] = compute_single(u_real, m);
92 else
93     % General complex case using addition formulas
94     [sn_real, cn_real, dn_real] = compute_single(u_real, m);
95     [sn_imag, cn_imag, dn_imag] = compute_single(u_imag, 1-m);
96
97     denom = cn_imag.^2 + m * sn_real.^2 .* sn_imag.^2;
98
99     sn = (sn_real .* dn_imag + 1i * cn_real .* dn_real .* sn_imag .* cn_imag) ./ denom;
100    cn = (cn_real .* cn_imag - 1i * sn_real .* dn_real .* sn_imag .* dn_imag) ./ denom;
101    dn = (dn_real .* cn_imag .* dn_imag - 1i * m * sn_real .* cn_real .* sn_imag) ./ denom;
102    ;
103 end
104
105 function [K, Kprime] = complete_elliptic_integrals(m)
106 % Compute complete elliptic integrals K(m) and K'(m) using AGM
107 if m == 0
108     K = pi/2;
109     Kprime = inf;
110 elseif m == 1
111     K = inf;

```

```

112     Kprime = pi/2;
113 else
114     % Compute  $K(m)$ 
115     [a_final, ~, ~, ~] = optimized_agm(m);
116     K = pi / (2 * a_final);
117
118     % Compute  $K'(m) = K(1-m)$ 
119     [a_final_prime, ~, ~, ~] = optimized_agm(1-m);
120     Kprime = pi / (2 * a_final_prime);
121 end
122 end
123
124 function [a_final, a_list, c_list, N] = optimized_agm(m)
125 % Optimized AGM iteration with precise convergence criteria
126 tol = 1e-16;
127 max_iter = 100;
128
129 a = 1.0;
130 b = sqrt(1 - m);
131 c = (a - b) / 2;
132
133 a_list = a;
134 c_list = c;
135 N = 0;
136
137 for iter = 1:max_iter
138     a_next = (a + b) / 2;
139     b_next = sqrt(a * b);
140     c_next = (a - b) / 2;
141
142     a_list(end+1) = a_next;
143     c_list(end+1) = c_next;
144     N = N + 1;
145
146     % Check multiple convergence criteria
147     if abs(c_next) < tol * a_next
148         break;
149     end
150     if abs(a_next - b_next) < tol * a_next
151         break;
152     end
153
154     a = a_next;
155     b = b_next;
156 end
157
158 a_final = a_list(end);
159 end
160
161 function [u_reduced, sign_sn, sign_cn] = reduce_argument_real(u, K)
162 % Reduce real argument to fundamental domain  $[0, K]$  with proper sign handling
163 sign_sn = 1;
164 sign_cn = 1;
165
166 if isinf(K) %  $m = 1$  case handled separately
167     u_reduced = u;
168     return;
169 end
170
171 % Reduce modulo  $4K$ 
172 u_mod = mod(u, 4*K);
173
174 % Handle negative arguments

```

```

175     if u_mod < 0
176         u_mod = u_mod + 4*K;
177     end
178
179     % Determine quadrant and reduce to [0, K]
180     if u_mod > 3*K
181         u_reduced = 4*K - u_mod;
182         sign_sn = -1;
183         sign_cn = -1;
184     elseif u_mod > 2*K
185         u_reduced = u_mod - 2*K;
186         sign_sn = -1;
187     elseif u_mod > K
188         u_reduced = 2*K - u_mod;
189         sign_cn = -1;
190     else
191         u_reduced = u_mod;
192     end
193
194     % Ensure u_reduced is in [0, K]
195     u_reduced = max(0, min(K, u_reduced));
196 end
197
198 function phi = backward_reconstruction(u, a_final, a_list, c_list, N, m)
199 % Improved backward reconstruction with error control
200
201 % Initial amplitude approximation
202 phi = 2^N * a_final * u;
203
204 % Backward iteration with careful handling
205 for i = N:-1:1
206     ratio = c_list(i) / a_list(i+1);
207
208     % Use alternative formula for better numerical stability
209     if abs(ratio) < 0.1
210         x = ratio * sin(phi);
211     else
212         x = sin(phi) * ratio;
213     end
214
215     % Clamp to avoid numerical issues
216     x = min(max(x, -1), 1);
217
218     % Improved reconstruction formula
219     phi_old = phi;
220     phi = (phi + asin(x)) / 2;
221
222     % Check for convergence in backward step
223     if abs(phi - phi_old) < 1e-15
224         break;
225     end
226 end
227
228 % Final refinement using Newton iteration if needed
229 if m > 1e-10 && m < 1 - 1e-10
230     phi = refine_amplitude(phi, u, m);
231 end
232 end
233
234 function phi_refined = refine_amplitude(phi, u, m)
235 % Refine amplitude using Newton iteration
236 max_refine = 3;
237 tol_refine = 1e-16;

```

```

238
239     for refine = 1:max_refine
240         sn_phi = sin(phi);
241         cn_phi = cos(phi);
242         dn_phi = sqrt(1 - m * sn_phi.^2);
243
244         % Compute the elliptic integral approximation
245         F_approx = elliptic_integral_approx(phi, m);
246
247         residual = F_approx - u;
248
249         if abs(residual) < tol_refine
250             break;
251         end
252
253         % Newton step: dF/dphi = 1/dn_phi
254         delta = residual * dn_phi;
255         phi = phi - delta;
256
257         % Keep phi in reasonable range
258         if phi > pi/2
259             phi = pi/2;
260         elseif phi < -pi/2
261             phi = -pi/2;
262         end
263     end
264
265     phi_refined = phi;
266 end
267
268 function F = elliptic_integral_approx(phi, m)
269 % Approximate elliptic integral using AGM (inverse of what we're computing)
270 if m == 0
271     F = phi;
272 elseif m == 1
273     F = asinh(tan(phi));
274 else
275     % Simple approximation - for refinement only
276     F = phi + (m/4) * (phi - sin(phi)*cos(phi));
277 end
278 end
279
280 function [sn, cn, dn] = series_expansion(u, m)
281 % Series expansion for very small arguments
282 u2 = u.^2;
283 u3 = u.^3;
284 u4 = u2.^2;
285 u5 = u2.*u3;
286 u6 = u3.^2;
287 u7 = u3.*u4;
288
289 % Taylor series expansions around u=0
290 sn = u - (1 + m)*u3/6 + (1 + 14*m + m^2)*u5/120 - ...
291     (1 + 135*m + 135*m^2 + m^3)*u7/5040;
292
293 cn = 1 - u2/2 + (1 + 4*m)*u4/24 - (1 + 44*m + 16*m^2)*u6/720;
294
295 dn = 1 - m*u2/2 + m*(4 + m)*u4/24 - m*(16 + 44*m + m^2)*u6/720;
296
297 % Ensure values are properly bounded
298 if isreal(u)
299     sn = min(max(sn, -1), 1);
300     cn = min(max(cn, -1), 1);

```

```
301     dn = min(max(dn, sqrt(1-m)), 1);  
302     end  
303 end
```

Listing A.1: `ellipj_from_scratch.m` — from-scratch AGM-based Jacobi elliptic function evaluator

A.2 Complete Filter Design Script

The following listing is the complete `Final_Elliptic_filter_design.m` script implementing the full design flow of Chapters 2–5.

```

1  % clc;
2  % clear;
3  % close all;
4  M = input("Enter the Filter Number : ");
5  delta1 = input(" Enter the value of Passband Tolerance : ");
6  delta2 = input(" Enter the value of Stopband Tolerance : ");
7  Tw = input(" Enter the value of transition width : ");
8  fs = input("Enter the value of Sampling Frequency : ")
9  Q = floor(M/11) % Quotient
10 R = rem(M,11) % Remainder
11 %% Filter 1 Calculations
12 % Calculation of Stopband Range for Filter 1
13 fs1 = 40 + 5*Q
14 fs2 = 70 + 5*Q
15 fprintf('Stopband Range of filter 1 should be: %.2f kHz to %.2f kHz\n', fs1, fs2);
16 % Calculation of Passband Range for Filter 1
17 fp1 = fs1 - Tw
18 fp2 = fs2 + Tw
19 fprintf('Stopband Range of filter 1 should be: 0 to %.2f kHz and %.2f kHz to inf\n', fp1, fp2
20 );
21 %% Bilinear Transformation - tan(pi*f/fs) for Filter 1
22 omegas1 = tan(pi*fs1/fs)
23 omegas2 = tan(pi*fs2/fs)
24 omegap1 = tan(pi*fp1/fs)
25 omegap2 = tan(pi*fp2/fs)
26 B = omegap2 - omegap1
27 omega0 = sqrt(omegap2*omegap1)
28 fprintf('Stopband Range of filter 1 should be: %.2f kHz to %.2f kHz\n', omegas1, omegas2);
29 fprintf('Stopband Range of filter 1 should be: 0 to %.2f kHz and %.2f kHz to inf\n', omegap1,
30 omegap2);
31 %% Mapping of BSP to Butterworth LPF - B*Omega/(omega0^2 - omega^2) for Filter 1
32 omegaLp1 = B*omegap1/(omega0^2 - omegap1^2)
33 omegaLp2 = B*omegap2/(omega0^2 - omegap2^2)
34 omegaLs1 = B*omegas1/(omega0^2 - omegas1^2)
35 omegaLs2 = B*omegas2/(omega0^2 - omegas2^2)
36 fprintf('Stopband Range of filter 1 should be: %.2f kHz to %.2f kHz\n', omegaLs1, omegaLs2);
37 fprintf('Stopband Range of filter 1 should be: 0 to %.2f kHz and %.2f kHz to inf\n', omegaLp1
38 , omegaLp2);
39 omegaLs = min(abs(omegaLs1),abs(omegaLs2))
40 % Calculation of order
41 k = 1/omegaLs
42 k_ = sqrt(1 - k^2)
43 epsilon = sqrt(2*delta1 - delta1^2)/sqrt(1 - 2*delta1 + delta1^2)
44 k1 = epsilon/sqrt(1/delta2^2 - 1)
45 k1_ = sqrt(1 - k1^2)
46 fprintf("Value of k : ")
47 disp(k)
48 fprintf("Value of kdass : ")
49 disp(k_)
50 fprintf("Value of k1 : ")
51 disp(k1)
52 fprintf("Value of k1dass : ")
53 disp(k1_)
54 %integral calculation
55 f1 = @(y) 1 ./ sqrt(1 - k^2 * sin(y).^2);
56 K = integral(f1,0,pi/2)

```

```

56 disp(K)
57 f2 = @(y) 1 ./ sqrt(1 - k_^2 * sin(y).^2);
58 K_ = integral(f2,0,pi/2)
59 disp(K_)
60 f3 = @(y) 1 ./ sqrt(1 - k1^2 * sin(y).^2);
61 K1 = integral(f3,0,pi/2)
62 disp(K1)
63 f4 = @(y) 1 ./ sqrt(1 - k1_^2 * sin(y).^2);
64 K1_ = integral(f4,0,pi/2)
65 disp(K1_)
66 f5 = inv_sc_elliptic(1/epsilon,k1_)
67 N = (K*K1_)/(K_*K1)
68 N = ceil(N)
69 fprintf(" Order of Filter is : ")
70 disp(N)
71
72 zeros = []; % Initialize
73
74 if mod(N,2) == 1
75     % --- For odd N ---
76     % o = 0, 2, 4, ..., (N-1)
77     o_vals = 0:2:(N-1);
78 else
79     % --- For even N ---
80     % o = 1, 3, 5, ..., (N-1)
81     o_vals = 1:2:(N-1);
82 end
83
84 for o = o_vals
85     [sn, cn, dn] = ellipj_from_scratch(o * K / N, k^2);
86     z_val = 1j ./ (k .* sn); % Zero locations
87     zeros = [zeros, z_val, -z_val]; % Add conjugate pair
88 end
89
90 fprintf('\nZeros:\n');
91 disp(zeros.');
92
93 %poles
94 f5 = inv_sc_elliptic(1/epsilon,k1_^2)
95 v = (K*f5)/(N*K1)
96
97 poles = [];
98
99 if mod(N,2) == 1
100     % N is odd
101     l_vals = 0:2:(N-1);
102 else
103     % N is even
104     l_vals = 1:2:(N-1);
105 end
106
107 for l = l_vals
108     [sn, cn, dn] = ellipj_from_scratch(l * K / N, k^2);
109     [sn_, cn_, dn_] = ellipj_from_scratch(v, k_^2);
110
111     % Pole formula
112     p_val = ((-1)*cn * dn * sn_ * cn_ + 1j * sn * dn_) / (1 - dn^2 * sn_^2);
113
114     % Add only one copy if real, else add conjugate pair
115     if abs(imag(p_val)) < 1e-12
116         poles = [poles, p_val]; % real pole → add once
117     else
118         poles = [poles, p_val, conj(p_val)]; % complex pole pair

```

```

119     end
120 end
121
122 fprintf('\nPoles:\n');
123 disp(poles.);
124 %% ----- Transfer Function H(s) -----
125 [num_lp, den_lp] = zp2tf(zeros.', poles.', 1);
126 Hlp = tf(num_lp, den_lp);
127 H0q = abs(polyval(num_lp,0) / polyval(den_lp,0)); % DC gain
128 num_scaled = num_lp / H0q; % scale numerator for |H(0)| = 1
129 Hlp = tf(num_scaled, den_lp);
130 disp('Butterworth Lowpass Transfer Function H(s) of filter 1:');
131 Hlp
132
133 %% ----Low pass Elliptic filter ----
134 figure;
135 bode(Hlp);
136 grid on;
137 title('Magnitude and Phase Response of Elliptic Filter H(s)');
138
139 %% ----- Pole-Zero Plot -----
140 figure;
141 pzmap(Hlp);
142 grid on;
143 title('Pole-Zero Map of Elliptic Filter H(s)');
144
145 %% ----- Frequency Response -----
146 figure;
147 freqs(num_scaled, den_lp);
148 title('Frequency Response |H(j\omega)| of Elliptic Filter');
149 xlabel('Frequency (rad/s)');
150 ylabel('Magnitude');
151 grid on;
152
153 %% ----- Bandstop Substitution s -> (B*s)/(s\Omega^2+0^2) -----
154 syms s z
155 qsubs_expr = (B*s)/(s^2 + omega0^2); % for First Filter
156 num_bp = poly2sym(num_scaled,s);
157 den_bp = poly2sym(den_lp,s);
158
159 qnum_bp_sub = expand(subs(num_bp, s, qsubs_expr));
160 qden_bp_sub = expand(subs(den_bp, s, qsubs_expr));
161 qHbp = qnum_bp_sub/qden_bp_sub;
162 fprintf('Analog Bandpass Transfer Function (First Filter) qHbp(s)\n');
163 qHbp
164 %% ----- Extract Analog Bandstop Coefficients -----
165 % Convert symbolic transfer function to numerator and denominator polynomials
166 [num_bs_analog, den_bs_analog] = numden(qHbp);
167
168 % Extract coefficients
169 num_bs_analog_coeff = sym2poly(num_bs_analog);
170 den_bs_analog_coeff = sym2poly(den_bs_analog);
171
172 % Normalize by leading denominator coefficient
173 k_analog = den_bs_analog_coeff(1);
174 num_bs_analog_coeff = num_bs_analog_coeff / k_analog;
175 den_bs_analog_coeff = den_bs_analog_coeff / k_analog;
176
177 fprintf('\n=== FILTER 1 ANALOG BANDSTOP COEFFICIENTS ===\n');
178 fprintf('Analog Numerator coefficients (s-domain): ');
179 fprintf('%.6f ', num_bs_analog_coeff);
180 fprintf('\n');
181 fprintf('Analog Denominator coefficients (s-domain): ');

```

```

182 fprintf('%.6f ', den_bs_analog_coeff);
183 fprintf('\n');
184
185 %% ----- Bilinear Transformation (s -> (z-1)/(z+1)) -----
186 qHbp_z = simplify(subs(qHbp, s, (z-1)/(z+1)));
187
188 disp('Discrete Transfer Function (First Filter) H(z):');
189 pretty(qHbp_z);
190
191 %% ----- Convert to numeric coefficients -----
192 [nz1, dz1] = numden(qHbp_z);
193 nz1 = sym2poly(expand(nz1));
194 dz1 = sym2poly(expand(dz1));
195 % --- Normalize by leading denominator coefficient (you already do this)
196 k12 = dz1(1);
197 nz1 = nz1 / k12;
198 dz1 = dz1 / k12;
199 disp('Numerator coefficients of analog BSF-I in z:');
200 disp(mat2str(nz1));
201 disp('Denominator coefficients of analog BSF-I in z:');
202 disp(mat2str(dz1));
203 fvtool(nz1,dz1)
204
205 %% Filter 2 Calculations
206 % Calculation of Stopband Range for Filter 2
207 fs11 = 170 + 5*R
208 fs22 = 200 + 5*R
209 fprintf('Stopband Range of filter 2 should be: %.2f kHz to %.2f kHz\n', fs11, fs22);
210 % Calculation of Passband Range for Filter 2
211 fp11 = fs11 - Tw
212 fp22 = fs22 + Tw
213 fprintf('Passband Range of filter 2 should be: 0 to %.2f kHz and %.2f kHz to inf\n', fp11,
    fp22);
214
215 %% Bilinear Transformation - tan(pi*f/fs) for Filter 2
216 omegas11 = tan(pi*fs11/fs)
217 omegas22 = tan(pi*fs22/fs)
218 omegap11 = tan(pi*fp11/fs)
219 omegap22 = tan(pi*fp22/fs)
220 B1 = omegap22 - omegap11
221 omega01 = sqrt(omegap22*omegap11)
222 fprintf('Stopband Range of filter 2 should be: %.2f kHz to %.2f kHz\n', omegas11, omegas22);
223 fprintf('Stopband Range of filter 2 should be: 0 to %.2f kHz and %.2f kHz to inf\n', omegap11,
    omegap22);
224
225 %% Mapping of BSP to Butterworth LPF - B*Omega/(omega0^2 - omega^2) for Filter 2
226 omegaLp11 = B1*omegap11/(omega01^2 - omegap11^2)
227 omegaLp22 = B1*omegap22/(omega01^2 - omegap22^2)
228 omegaLs11 = B1*omegas11/(omega01^2 - omegas11^2)
229 omegaLs22 = B1*omegas22/(omega01^2 - omegas22^2)
230 fprintf('Stopband Range of filter 2 should be: %.2f kHz to %.2f kHz\n', omegaLs11, omegaLs22)
    ;
231 fprintf('Stopband Range of filter 2 should be: 0 to %.2f kHz and %.2f kHz to inf\n',
    omegaLp11, omegaLp22);
232 omegaLs1 = min(abs(omegaLs11),abs(omegaLs22))
233 % Calculation of order
234 k = 1/omegaLs1
235 k_ = sqrt(1 - k^2)
236 epsilon = sqrt(2*delta1 - delta1^2)/sqrt(1 - 2*delta1 + delta1^2)
237 k1 = epsilon/sqrt(1/delta1^2 - 1)
238 k1_ = sqrt(1 - k1^2)
239 fprintf('Value of k : ')
240 disp(k)

```

```

241 fprintf("Value of kdass : ")
242 disp(k_)
243 fprintf("Value of k1 : ")
244 disp(k1)
245 fprintf("Value of k1dass : ")
246 disp(k1_)
247 %integral calculation
248 f1 = @(y) 1 ./ sqrt(1 - k^2 * sin(y).^2);
249 K = integral(f1,0,pi/2)
250 disp(K)
251 f2 = @(y) 1 ./ sqrt(1 - k^2 * sin(y).^2);
252 K_ = integral(f2,0,pi/2)
253 disp(K_)
254 f3 = @(y) 1 ./ sqrt(1 - k1^2 * sin(y).^2);
255 K1 = integral(f3,0,pi/2)
256 disp(K1)
257 f4 = @(y) 1 ./ sqrt(1 - k1_^2 * sin(y).^2);
258 K1_ = integral(f4,0,pi/2)
259 disp(K1_)
260 f5 = inv_sc_elliptic(1/epsilon,k1_)
261 N2 = (K*K1_)/(K*K1)
262 N2 = ceil(N2)
263 fprintf(" Order of 2nd Filter is : ")
264 disp(N2)
265 % Calculation of Poles
266 zeros1 = []; % Initialize
267
268 if mod(N,2) == 1
269     % --- For odd N ---
270     % o = 0, 2, 4, ..., (N-1)
271     o_vals1 = 0:2:(N2-1);
272 else
273     % --- For even N ---
274     % o = 1, 3, 5, ..., (N-1)
275     o_vals1 = 1:2:(N2-1);
276 end
277
278 for o = o_vals1
279     [sn1, cn1, dn1] = ellipj_from_scratch(o * K / N2, k^2);
280     z_val1 = 1j ./ (k .* sn1); % Zero locations
281     zeros1 = [zeros1, z_val1, -z_val1]; % Add conjugate pair
282 end
283
284 fprintf('\nZeros:\n');
285 disp(zeros1.');
286
287 %Calculation of poles
288 f5 = inv_sc_elliptic(1/epsilon,k1_^2)
289 v1 = (K*f5)/(N2*K1)
290
291 poles1 = [];
292
293 if mod(N2,2) == 1
294     % N is odd
295     l_vals = 0:2:(N2-1);
296 else
297     % N is even
298     l_vals = 1:2:(N2-1);
299 end
300
301 for l = l_vals
302     [sn1, cn1, dn1] = ellipj_from_scratch(l * K / N2, k^2);
303     [sn_, cn_, dn_] = ellipj_from_scratch(v, k^2);

```

```

304
305     % Pole formula
306     p_val = ((-1)*cn1 * dn1 * sn_ * cn_ + 1j * sn1 * dn_) / (1 - dn1^2 * sn_^2);
307
308     % Add only one copy if real, else add conjugate pair
309     if abs(imag(p_val)) < 1e-12
310         poles1 = [poles1, p_val];
311     else
312         poles1 = [poles1, p_val, conj(p_val)]; % complex pole pair
313     end
314 end
315
316 fprintf('\nPoles:\n');
317 disp(poles1.);
318 %% ----- Transfer Function H(s) -----
319 [num_lp, den_lp] = zp2tf(zeros1.', poles1.', 1);
320 Hlp = tf(num_lp, den_lp);
321 H0q = abs(polyval(num_lp,0) / polyval(den_lp,0)); % DC gain
322 num_scaled = num_lp / H0q; % scale numerator for |H(0)| = 1
323 Hlp = tf(num_scaled, den_lp);
324 disp('Butterworth Lowpass Transfer Function H(s) of filter 2:');
325 Hlp
326
327 %% ----Low pass Elliptic filter ----
328 figure;
329 bode(Hlp);
330 grid on;
331 title('Magnitude and Phase Response of Elliptic Filter H(s)');
332
333 %% ----- Pole-Zero Plot -----
334 figure;
335 pzmap(Hlp);
336 grid on;
337 title('Pole-Zero Map of Elliptic Filter H(s)');
338
339 %% ----- Frequency Response -----
340 figure;
341 freqs(num_scaled, den_lp);
342 title('Frequency Response |H(j\omega)| of Elliptic Filter');
343 xlabel('Frequency (rad/s)');
344 ylabel('Magnitude');
345 grid on;
346
347 %% ----- Bandstop Substitution s -> (B*s)/(s\Omega^2+0^2) -----
348 syms s z
349 qsubs_expr = (B1*s)/(s^2 + omega01^2);
350 num_bp = poly2sym(num_scaled,s);
351 den_bp = poly2sym(den_lp,s);
352
353 qnum_bp_sub = expand(subs(num_bp, s, qsubs_expr));
354 qden_bp_sub = expand(subs(den_bp, s, qsubs_expr));
355 qHbp = qnum_bp_sub/qden_bp_sub;
356 fprintf('Analog Bandpass Transfer Function (2nd Filter) qHbp(s)\n');
357 qHbp;
358 %% ----- Extract Analog Bandstop Coefficients -----
359 % Convert symbolic transfer function to numerator and denominator polynomials
360 [num_bs_analog, den_bs_analog] = numden(qHbp);
361
362 % Extract coefficients
363 num_bs_analog_coeff = sym2poly(num_bs_analog);
364 den_bs_analog_coeff = sym2poly(den_bs_analog);
365
366 % Normalize by leading denominator coefficient

```

```

367 k_analog = den_bs_analog_coeff(1);
368 num_bs_analog_coeff = num_bs_analog_coeff / k_analog;
369 den_bs_analog_coeff = den_bs_analog_coeff / k_analog;
370
371 fprintf('\n=== FILTER 1 ANALOG BANDSTOP COEFFICIENTS ===\n');
372 fprintf('Analog Numerator coefficients (s-domain): ');
373 fprintf('%0.6f ', num_bs_analog_coeff);
374 fprintf('\n');
375 fprintf('Analog Denominator coefficients (s-domain): ');
376 fprintf('%0.6f ', den_bs_analog_coeff);
377 fprintf('\n');
378 %% ----- Bilinear Transformation (s -> (z-1)/(z+1)) -----
379 qHbp_z = simplify(subs(qHbp, s, (z-1)/(z+1)));
380
381 disp('Discrete Transfer Function (2nd Filter) H(z):');
382 pretty(qHbp_z);
383
384 %% ----- Convert to numeric coefficients -----
385 [nz11, dz11] = numden(qHbp_z);
386 nz11 = sym2poly(expand(nz11));
387 dz11 = sym2poly(expand(dz11));
388 % --- Normalize by leading denominator coefficient
389 k11 = dz11(1);
390 nz11 = nz11 / k11;
391 dz11 = dz11 / k11;
392 disp('Numerator coefficients of analog BSF-II in z:');
393 disp(mat2str(nz11));
394 disp('Denominator coefficients of analog BSF-II in z:');
395 disp(mat2str(dz11));
396 fvtool(nz11,dz11)
397
398 %Cascade of 2 Band stop Filter
399 x = conv(nz1,nz11)
400 y = conv(dz1,dz11)
401 disp('Numerator coefficients of analog Multi - BSF in z:');
402 disp(mat2str(x));
403 disp('Denominator coefficients of analog Multi - BSF in z:');
404 disp(mat2str(y));
405 fvtool(x,y)

```

Listing A.2: Final_Elliptic_filter_design.m — complete multi-band-stop elliptic filter design script